IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 5 February 2025, accepted 2 March 2025, date of publication 11 March 2025, date of current version 19 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3550123

RESEARCH ARTICLE

Evaluating the Impact of Pair Documentation on Requirements Quality in Agile Software Development

NOSHEEN QAMAR[1], NOSHEEN SABAHAT[2], AND AMIR MOSAVI[3,4,5]

[1]Department of Software Engineering, University of Management and Technology, Lahore 54000, Pakistan
[2]Department of Computer Science, Forman Christian College (A Chartered University), Lahore 54000, Pakistan
[3]Ludovika University of Public Service, 1083 Budapest, Hungary
[4]John von Neumann Faculty of Informatics, Obuda University, 1034 Budapest, Hungary
[5]Abylkas Saginov Karaganda Technical University, Karaganda 100000, Kazakhstan

Corresponding authors: Nosheen Sabahat (nosheensabahat@fccollege.edu.pk) and Amir Mosavi (amir.mosavi@nik.uni-obuda.hu)

ABSTRACT A fundamental deliverable of the requirements engineering process is the software requirements specification (SRS) document. A reliable SRS ensures that the final product aligns with the stakeholder's expectations. Yet poorly documented requirements may lead to significant project risks, which include delays, cost overruns, and product failure. Currently, delivering a precise SRS remains a complex task which sometimes result in miscommunication and incomplete information. Such challenges are particularly seen in agile development where the iterative nature of projects requires continuous refinement of the requirements. Consequently, this empirical research aims to evaluate the impact of pair documentation on the quality and effectiveness of SRS documents within agile development teams. Twenty pairs of documentation writers worked into two groups where one group used the pair documentation, i.e., the experimental group, and the other one used conventional documentation i.e., the control group. Both groups were tasked with producing SRS documents for the same project. The outcomes are evaluated against three key criteria of completeness, ambiguity, and consistency of the documents. Our preliminary analysis indicates that the experimental group demonstrated an improvement in document quality with fewer ambiguities, more comprehensive coverage of requirements, and greater consistency across sections. Our research further suggests that collaborative efforts in pair documentation not only enhance the clarity and precision of SRS documents but also may mitigate risks associated with possible miscommunications. In addition, the findings of this study emphasize the potential of pair documentation as a strategy to deliver higher quality SRS documents in dynamic project environments.

INDEX TERMS Agile development, pair documentation, pair programming, requirements engineering, soft computing, software quality, software requirements, software requirements specification.

I. INTRODUCTION

Over the past few years, software development has become an integral part of the software industry. Software is the most important component of almost every field and the success of every business depends on the usage of successful software. However, even though the software has become ubiquitous, there is no key to success in the development of high-quality software. Factors like high-end resources, a better work environment, technical expertise, the right software development process, growing complexities, market trends, changing technology, etc. are all influencing the success of the software [1]. There are various challenges faced by software development teams. For instance, the success of software nowadays is highly dependent on the methodologies used for software development. Furthermore, there is a dire need to integrate various programs, within a limited budget and timelines. In such scenarios, agile methodologies

The associate editor coordinating the review of this manuscript and approving it for publication was Antonio Piccinno.

are considered more successful and popular in improving software quality [2]. Requirements Engineering (RE) is the first phase in the traditional software development process whereas, in agile development, it continues throughout the process [3]. Literature reveals that an unsuccessful RE process is due to multiple reasons like lack of user involvement, missing or incomplete requirements, vague, ambiguous, changing requirements, etc. This official Software Requirements Specification (SRS) document is a contract among the stakeholders [11]. The success of a project is highly dependent on the outcome of the RE process, i.e., good quality SRS documents [11]. The studies conducted by the Standish Group [12] found that the project success rate has been 29% or less in the past twenty years. The top factors behind it are related to requirements such as lack of user involvement, incomplete/changing requirements and specifications, and lack of clear statement of requirements. A project can only be successful if it is given the right level of attention to clarify the key requirements. It may take from 25% to 50% of the total project time [13], [14] to develop a good-quality SRS document. Therefore, more research is needed in the area of requirements engineering to make this process more effective and to produce improved quality SRS documents. Our research aims to fill this gap by proposing a new approach to the software requirements documentation process. The term ''Pair Documentation'' was first coined by Scott Ambler in his book on Agile Modelling [15]. In pair documentation, two people work simultaneously on the same requirements document. This concept is similar to the well-known concept of pair programming. This study empirically assesses the usefulness of pair documentation over conventional documentation on the same project. An experiment is designed and conducted using a control group and an experimental group, each consisting of 10 teams working on the requirements document. The experimental group teams work using a pair documentation approach whereas the control group teams work using conventional documentation. The resultant SRS of the projects is then assessed based on the quality metrics of the two groups of teams. The rest of the paper is structured as follows. The next section summarizes related work done in this area. Section III describes our proposed approach and a detailed description of the design. Section IV presents various threats to the validity of this research. Finally, Section V summarizes some major contributions of this research and presents some directions for future work.

II. LITERATURE REVIEW

The effectiveness of the various steps in software development depends on the quality of the requirements specification. Therefore, it is important to assess the quality of SRS [16]. In the process of automating and developing software, the first and most important step is to identify the requirements and document them in the software requirement specification (SRS). The importance of software requirement specification (SRS) lies in documenting the essential requirements of software. While many studies have examined the quality of SRS, they often lack proper organization and representation of functional requirements and evaluate the quality of SRS based on the properties of completeness, correctness, preciseness, and consistency [4]. The analysis of SRS documentation requires the extraction of accurate information, which can be challenging as the complexity of software systems continues to increase. Tasks that target specific types of information within the SRS require experts in the field, and it can be costly and overwhelming for a limited number of experts [5]. Recent research has found that development teams often neglect proper documentation due to tight schedules or resource constraints. This can lead to difficulties in maintaining the software system. To address this issue, a mechanism is needed to automatically detect and update the software requirement specification documentation [6]. Although it is common for SRS to contain ambiguities, eliminating them is crucial and can greatly impact the success of the software development process. Literature reveals that authors have discussed ways to prevent, detect, and fix ambiguities in the requirements document [7]. Moreover, a value-oriented review (VOR) is also discussed which involves identifying core values based on the SRS and finding defects that go against these values [8]. To avoid such ambiguities, it is important to have clear requirements for software development. The usage of automated tools is suggested to analyze requirements to save time and effort [9]. The importance of the RE process by surveying identified that many fresh graduates have a deep interest in the field of RE and keep learning more about it [10]. Although Natural Language Processing (NLP) techniques have been suggested as a semi-automatic way of optimizing requirement specifications, they have not been widely adopted [17]. Moreover, assessing the quality of Software Requirement Specification (SRS) automatically is challenging, as it requires advanced algorithms to extract features, interpret context, formulate metrics, and document shortcomings [18]. The accuracy, consistency, and completeness of software quality depends on the accuracy of the requirement analysis phase which is the basis of software system development. However, manual preparation of software requirements specifications often leads to inconsistency with a business description, communication difficulties with business personnel, and low work efficiency [19]. It is being observed that involving users thoroughly in software requirements specification and design process will lead to the creation of dependable and acceptable software systems [20]. Literature reveals that the concept of working in pairs has been observed to be very effective. This can be done in various phases of the software development life cycle (SDLC) phases such as programming in pairs, designing in pairs, testing in pairs, etc. Hence the work can be done effectively and efficiently. Pair programming is a collaborative approach in which two people work closely on the same task [21], [22]. While working together, the roles can be shifted among the collaborators. In this way, the work done will be efficient and effective [23]. A lot of research work has been done on pair programming [24], [25], [26]. In one of

TABLE 1. SWOT analysis of the related work.

Reference	Strengths	Weaknesses	Opportunities	Threats
J. Chong and R. Siino (2006) [23]	Explore interruption patterns between programmers working as solo and in pair	Observed teams for only 40 hours Limited interruption types i.e. five were discussed	The experiment can be replicated with diverse interruption types and a higher number of professionals	People with different personal behaviors have different levels of focus and attention to detail. Gender can affect the results
P. Baheti, E. Gehringer and D. Stotts (2002) [24]	Compare different working arrangements of student teams developing object-oriented software	Pairs who were students having no professional or dealing experience working from different locations	The experiment can be redone with the people of a large organization having its offices at different locations	Internet connectivity, the tools for collaboration, time zone, language, and culture can affect the results
K. M. Lui and K. C. Chan (2006) [28]	Observe the relation between programming productivity and human experience	The research results were based on the assignment work given to the students	The framework of this research can be used to improve overall productivity and optimize resources	Factors such as pair jelling, albeit were not fully controlled or were not measured in their experiments
Ö. Demir and S. S. Seferoglu (2020) [29]	This study compares solo and pair programming in terms of flow experience, coding quality, and coding achievement	The study does not evaluate computer programs in terms of originality, creativity, the function of code	This study can be replicated using a true experimental design, with a larger study group, a different coding environment, and a longer intervention period	The students in the pair programming group were continuously experiencing a new groupmate every week which can affect the results
N. Qamar and A. A. Malik (2019) [30]	This study examined the utility of an unconventional testing technique called pair testing	Only the black testing technique was explored in this experiment	Other aspects of quality like the severity of defects identified may be considered	This was a pilot study involving only 2 professionals and 4 student teams
G. Canfora, A. et al. (2006) [31]	This study compared pair designing with solo members	measured quality by rating, which is a subjective measure	The new design can be used to measure the functionality of the artifacts	subjects with high motivation showed a great interest can influence the results

their research, Kude et al. [22] discussed the effectiveness of pair programming and stated that instead of solo programmers, the work done in pairs proved to be more effective. Furthermore, Chong et al. [23] have presented a comparison of paired and solo programmers. They presented the analysis of 242 interruptions and discussed team configurations as well. They have suggested pair programming to support better. Vera et al. [27] presented an exploratory investigation that demonstrated a statistically significant difference in product quality between teams composed of individuals with compatible roles and those that are randomly integrated. In one of their research, Lui et al. [28] examined the productivity of pair programmers via a case study by doing repeat programming and reached various conclusions regarding novice-novice pairs against expert-expert pairs and solos as well. In addition to this, a comparison of Solo and Pair Programming has been conducted by Omer et al. in terms of Flow Experience, Coding Quality, and Coding Achievement [29]. After conducting several solo and group experiments, it was concluded that pair programming is effective to be used in educational settings. An empirical study was conducted by Qamar et al. [30] to investigate the usefulness of pair testing. The productivity of experimental groups was compared which revealed that the quality of the work was better in the case of pair testing. In another research conducted at the University of Castilla-La Mancha in Spain [31], the difference between solo and pair designing was studied. The results revealed that a good quality design document was produced as a result of pair designing. However, it has been observed that pair designing in various projects for experimentation took more time to complete the designing phase as compared to solo work. Swamidurai et al. [32] have examined the influence of pairing on various phases of software development. They assessed whether pairing is necessary for all phases by examining its impact on each phase of the SDLC. In a comprehensive study involving students as participants, they concluded that pairing is equally advantageous during the design and testing phases as it is during the coding phase. In this work, we evaluated the concept of pair documentation in which we conducted an experiment on one project and there was a single comparison conducted among a single person versus a single pair of participants for requirements specification. The results were gathered to check the effectiveness of the pair documentation approach. Despite the importance of pairing and homogeneity between team members [28], [29], [33], [34], [35], the concept of pairing in requirements engineering activities like documentation has not been addressed very well. Moreover, the usefulness and efficacy of working in pairs while programming, testing, and designing encouraged us to take the initiative of testing the practicality of pair documentation. Table 1 summarizes SWOT i.e., the strengths, weaknesses, opportunities, and threats analysis of the related work in this area.

III. RESEARCH METHODOLOGY

There are different phases involved in our experiment. Fig. 1 shows our proposed research methodology, and the following sections provide the details of each phase.

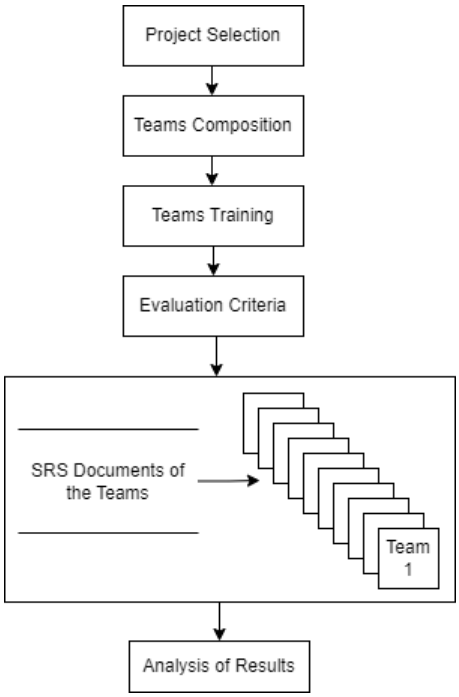

FIGURE 1. Research methodology.

A. PROJECT SELECTION

The first step in this research was an appropriate project selection. The project we used for this experiment is a utility bill payment system (UBPS) that records all the transactions booked by call center agents. UBPS stores three different types of utility bill transactions i.e., booked, auto-debit, and mobile. Booked transactions need to be booked explicitly through a call center agent each time a utility bill needs to be paid. Auto-debit transactions, on the other hand, are booked once only through a call center agent. Once booked, the bill automatically gets paid every month as per the defined schedule. These transactions apply to utility bills but not to mobile prepaid bills. Mobile transactions are used to pay mobile cellular companies' bills. Once a bill transaction has been booked, it must be approved by the call center administrator. After a call center administrator approves a transaction, another call center administrator verifies it and annotates it with her remarks. Finally, the bill is marked by an online authentication (OLA) officer who pays the bill and sends an SMS to the customer who booked the transaction. The main reason for selecting this system was one author's in-depth familiarity with the domain of billing systems. This in-depth familiarity is needed later in the experiment when this author plays the role of a client providing requirements.

B. TEAM COMPOSITION

In this study, 20 teams from a private university participated in the experiments. All the teams were assigned a term project-based task and were asked to write down the requirements of the same system i.e. UBPS. Teams were divided into two groups i.e. Experimental and Control group. The experimental group (10 teams having 20 participants) followed pair documentation i.e., one member documented the requirements and the other monitored the quality of those requirements). On the other hand, the controlled group having the same number of teams (10 teams having two participants in each team) worked individually following the conventional approach. The documentation work for the controlled group teams was divided into two parts and each member needed to work on his part alone. Each team contained people of the same level i.e. qualifications, experience, rank, and level of expertise. All teams consisted of students who were in their last semester. Furthermore, the teams being made in pairs were also uniform in terms of knowledge and expertise and none of them is superior to the other i.e. in each pair, both participants must not be experienced, and only one of them can have some experience of internship or freelancing and special interest in Requirements Engineering. Additionally, randomization was used to ensure an unbiased selection of participants. Table 2 shows the demographics of the teams who participated in this research.

TABLE 2. Team demographics.

Demographics	Details
Category/Level	Undergraduate
Semester	8th Semester
Degree	BSCS
Course	Advance Software Engineering
University	Pakistani Private University
Age Range	20-22
Assignment	Term Project

C. TEAM TRAINING

Before the teams started actual documentation, a training session was organized to deliver the requirements and guidelines of the experiment. According to the discussed guidelines, all requirements had to be written in the natural language (English) and at the same level with every minute detail in them. Every requirement will be documented in a manner that every requirement will have its unique requirement ID. All the functional requirements will have requirement ID as Req-001 onward and all the Non-Functional requirements will have requirement ID as Req-N-01 and so on. Requirements will be documented using MS Word (Fig 2 shows sample output as per the given guidelines). In addition to this, every member was responsible for his/her work, and they were not allowed to share any material related to the project with anyone. To get the requirements, one of the authors portrayed a client to generate a reference SRS which was not shared with any team but was only to do the comparison at the end of the experimental

TABLE 3. Sample list of ambiguous words.

Ambiguous Words						
Accommodate	Adequate	Minimize	Often	If necessary	Appropriate	Some
As a minimum	Easy	Etc.	Sufficient	Effective	Normal	Sometime
Robust	Can	Someone	May	Therefore	Capability to	Maximize

phase. The training was then provided to all the participants of the control and experimental groups, and everyone attended those sessions together to receive the same information together. These training sessions last over an hour consisting of 40 minutes to discuss the project and its requirements in detail and 20 minutes for the Q&A session. These training sessions were conducted not just to describe the project in brief, but they were productive in a manner that the proper training agenda was explained, functional and non-functional requirements were clarified, and the session also depicted how to write clear, complete, and concise requirements.

D. EVALUATION CRITERIA

The next phase of this research was evaluation criteria to answer the following research question:

RQ1: Which group produced a better-quality SRS document?

The answer to the research question was found by measuring the quality of SRS documents in terms of completeness, ambiguity, and consistency. The completeness was measured in terms of functional and non-functional requirements completeness. Functional requirements describe the functionality of a system in detail and mention how the system should behave in different situations. Non-functional requirements, on the other hand, deal with different quality or level-of-service aspects such as reliability, portability, usability, and performance. Completeness was judged by comparing requirements documented by the teams with the reference SRS document. Therefore, the criteria of completeness can be written as:

$$\begin{aligned} &Completeness \\ &= Total\ Requirements\ written\ by\ group/ \\ &\quad Total\ number\ of\ referenced\ Requirements \times 100\% \end{aligned}$$

For ambiguity and consistency, SRS documents were judged by looking at the presence of ambiguity in the listed requirements and inconsistency in SRS documents. A requirement is considered ambiguous if it has multiple interpretations. Even though there are different types of ambiguities e.g. lexical, semantic, referential, etc. [36], we focused on lexical ambiguities only. We have studied different quality assessment options [37], [38], [39]. After that, we selected an automated ambiguity detector tool [40] that was used to detect the presence of lexical ambiguities in the SRS documents. This tool takes input from the requirements and configuration files. The requirements file contains a list of functional and non-functional requirements whereas the configuration file includes a list of some ambiguous words as shown in Table 2. This list is used as a reference point for ambiguity detection in SRS. Therefore, ambiguity can be measured as:

$$\begin{aligned} &Ambiguity \\ &= Ambiguity\ calculated\ by\ tool/Total\ Ambiguity \times 100\% \end{aligned}$$

We measured the inconsistency in requirements manually comparing every documented requirement with every other documented requirement in the same SRS. Therefore, inconsistency can be calculated as:

$$\begin{aligned} &Inconsistency \\ &= Total\ Inconsistent\ Requirements\ Identified\ by\ the\ Experts \end{aligned}$$

Fig. 3 shows the automated ambiguity detector tool used to detect the presence of any lexical ambiguities in the SRS. It takes as input the requirements file consisting of all functional and non-functional requirements and the configuration file consisting of a list of ambiguous words.

Table 3 shows the list of ambiguous words (a sample is shown) which acts as a reference point for ambiguity detection. Moreover, while parsing requirements, it checks for the presence of all these words as mentioned in Table 3. After the parsing of the entire requirements, the results are generated as an output file. A sample output generated by this tool can be seen in Fig. 2.

Table 4 shows the details of the experimental attributes. According to it, there were a total of 126 requirements out of which 100 are functional and 26 are non-functional requirements. This experiment involved a total of forty BS(CS) final-year students who were divided into twenty teams (10 teams of the experimental group and 10 teams of the control group). It took almost 2 months to complete this experiment from planning to results compilation and analysis, out of that 14 working days were given to students to write their SRS documents. All the teams worked in parallel after this training session was over, for almost 14 working days to compile the SRS document for the system. The SRS document was taken from all the teams after the cut-off date.

E. ANALYSIS OF RESULTS

The quality of SRS documents was compared between the two groups. To prevent bias in assessment, the evaluators of the SRS were blinded to the documentation. Tables 5 and 6

This file contains the ambigious requirements of Team 1 (Experimental Group)

FR_001 Every User should access the application using authorized username and password. (ambigious word: and)

FR_002 User should enter the username and password on login page. (ambigious word: and)

FR_003 User should be able to Pay the bill. (ambigious word: be able to)

NFR_001 Only authorized user should be login to application with username and password. (ambigious word: and)

NFR_002 The messages should be encrypted for login communications, so others cannot get username and password from those messages. (ambigious word: can, and)

NFR_003 The system shall accommodate many users during the peak usage time. (ambigious word: accommodate)

Total No of Requirements in SRS of Team 1 : 82

Total No of Functional Requirements in SRS of Team 1 : 68

Total No of Non-Functional Requirements in SRS of Team 1 : 14

Total No of Ambigious Requirements in SRS of Team 1 : 14

Total No of Ambigious Functional Requirements in SRS of Team 1 : 6

Total No of Ambigious Non-Functional Requirements in SRS of Team 1 : 8

FIGURE 2. Sample output.

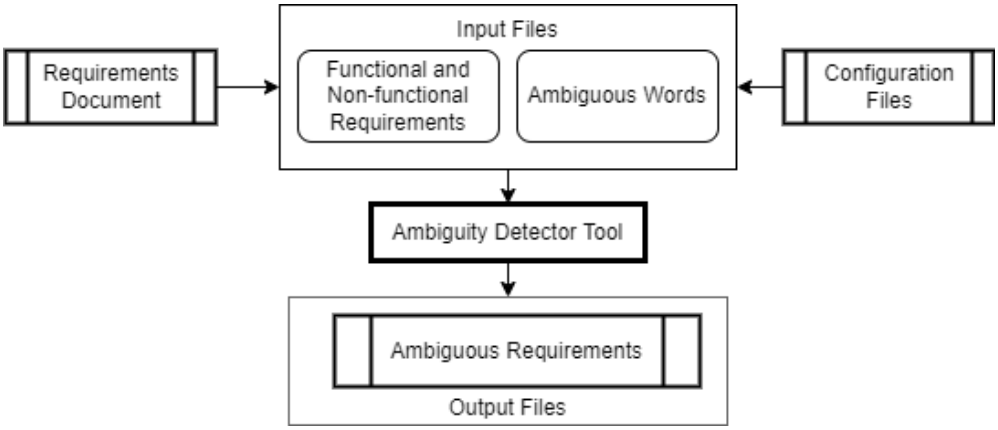

FIGURE 3. Ambiguity detector tool.

TABLE 4. Experimental details.

Attributes	Description
Domain of Reference SRS document	Online Transactions
Total Requirements in Reference SRS document	126 (Functional=100, Non-Functional=26)
Participants Involved	40 Final Year Students of BS(CS)
No. of Teams	20 (2 participants per team = Total 40)
Time allowed to complete the task/Total time	14 working days/ 2 Months
Training duration	120 minutes (60 Minutes for each group)
Evaluation Criteria/Metrics used for document quality	Completeness in requirements, % Ambiguity, and % Inconsistency

show the summary of results concerning the completeness, ambiguity, and consistency of the SRS document generated by the experimental and control groups, respectively. It can be observed that almost all the teams of the experimental group

TABLE 5. Comparison concerning completeness, ambiguity, and consistency for experimental group teams.

Sr#	Teams	Experimental Group Teams						
		Completeness			Ambiguity (%)		Consistency (%)	
		TR	FR	NFR	NAR (%)	AR (%)	CR (%)	ICR (%)
1	Team 1	82	68	14	83	17	93	7
2	Team 2	96	81	15	75	25	81	19
3	Team 3	55	43	12	68	32	73	27
4	Team 4	105	89	16	89	11	67	33
5	Team 5	68	57	11	63	37	58	42
6	Team 6	89	72	17	92	8	87	13
7	Team 7	98	84	14	58	42	61	39
8	Team 8	101	91	10	87	13	83	17
9	Team 9	97	84	13	81	19	76	24
10	Team 10	69	57	12	76	24	68	32
	Average	86	73	13	77	23	75	25

TR=Total Requirements, FR=Functional Requirements, NFR=Non-Functional Requirements, AR=Ambiguous Requirements, NAR=Non-Ambiguous Requirements, CR=Consistent Requirements, ICR=Inconsistent Requirements

TABLE 6. Comparison concerning completeness, ambiguity, and consistency for control group teams.

Sr#	Teams	Control Group Teams						
		Completeness			Ambiguity (%)		Consistency (%)	
		TR	FR	NFR	NAR (%)	AR (%)	CR (%)	ICR (%)
1	Team 1	73	64	9	70	30	82	18
2	Team 2	85	71	14	68	32	72	28
3	Team 3	93	79	14	57	43	85	15
4	Team 4	62	57	5	89	11	62	38
5	Team 5	95	83	12	78	22	74	26
6	Team 6	49	42	7	66	34	65	35
7	Team 7	89	76	13	91	9	75	25
8	Team 8	102	87	15	87	13	79	21
9	Team 9	63	54	9	53	47	68	32
10	Team 10	88	73	15	69	31	59	41
	Average	80	69	11	73	27	72	28

TR=Total Requirements, FR=Functional Requirements, NFR=Non-Functional Requirements, AR=Ambiguous Requirements, NAR=Non-Ambiguous Requirements, CR=Consistent Requirements, ICR=Inconsistent Requirements

(following pair documentation) were able to gather/complete more requirements as compared to the control group (traditional approach). For instance, Team 1 of the experimental group gathered 82 out of 126 requirements, out of which 68 out of 100 were Functional Requirements (FR) and 14 out of 26 were Non-Functional Requirements (NFR). On the contrary, Team 1 of the control group gathered 74 out of 126 requirements from which 64 were FR and 9 were NFR. The same is the case with the rest of the teams. This shows the experimental group to be more productive in providing a complete SRS as both the groups were working on the same project and requirements were given in the same time frame providing the answer to our research question (RQ1). Similarly, in terms of quality, it is evident that experimental group teams gathered more ambiguous and consistent requirements than the control group teams. For example, in the case of experimental group teams, 9 out of 10 teams were able to find more non-ambiguous requirements (NAR) and consistent requirements (CR) as compared to control group teams. The average results for the experimental teams for NAR and CR are 77 and 75 whereas in the case of the control group, these numbers are 73 and 72, respectively. These findings give us the answer to our research question.

Figs. 4 and 5 show the comparisons in terms of identification/completeness of functional requirements (FR) and non-functional requirements (NFR) by the experimental and control groups. It can be seen that in the case of FR 7 (except teams 3, 5, and 10) out of 10 teams and for NFR 6 (except teams 3, 5, 8, and 10) out of 10 teams of the experimental group collected more requirements as compared to the control group teams, hence answering the question of our research.

Fig. 6 shows the percentage of ambiguous requirements collected by the teams of experimental and control groups. It can be seen that the percentage of ambiguous requirements gathered by the teams of the control group is more

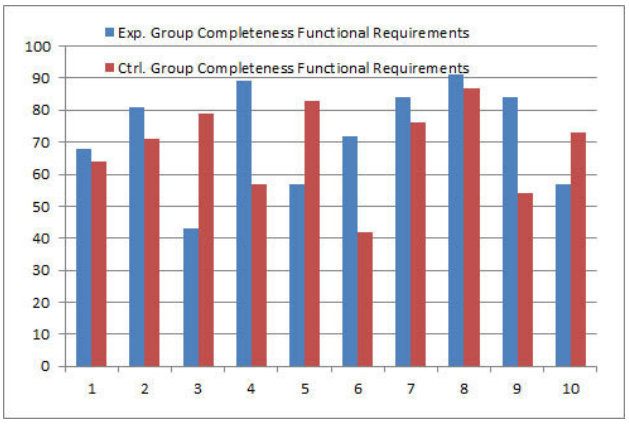

FIGURE 4. Comparison in terms of completeness of functional requirements.

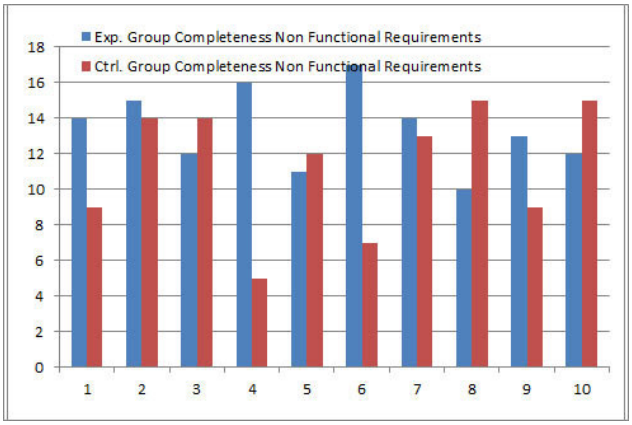

FIGURE 5. Comparison in terms of completeness of non-functional requirements.

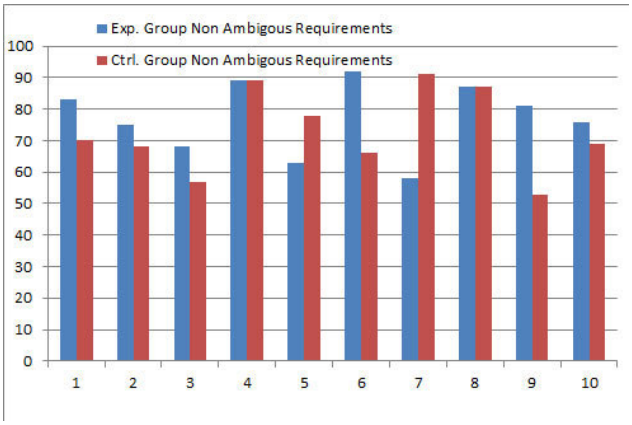

FIGURE 6. Comparison in terms of ambiguous requirements.

than the ambiguity value for experimental group teams. 8 (except 5 and 7) out of 10 teams belonging to the experimental group produced more or equal (in the case of teams 4 and 8) non-ambiguous requirements. Fig. 7 shows the comparison of the SRS documents in terms of the consistency of requirements. It is evident from the graph that all the teams

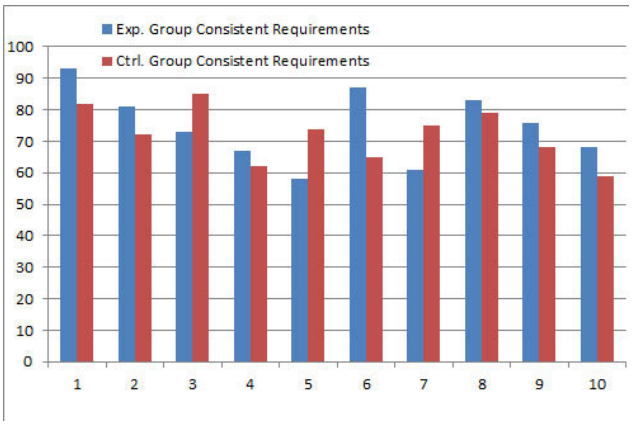

FIGURE 7. Comparison in terms of consistent requirements.

except teams 3, 5, and 7 of the experimental group gathered more consistent requirements. Therefore, these comparisons clearly show that the quality of requirements gathered by the experimental group teams using pair documentation is far better than those working alone. These results answer the research question of our research.

Fig. 8 represents the average results gathered from all ten teams from each group. It is very clear from the figure that the experimental teams elicited more requirements (for all three cases i.e. total requirements, functional requirements, and non-functional requirements) as compared to the control group teams acknowledging RQ1. The same is true for ambiguity as experimental teams produced more non-ambiguous and less ambiguous requirements than control group teams. In the case of consistency, the results represent the same proportions as those of ambiguity, replying to the RQ1.

As is evident from the above tables and figures, the teams from the experimental group (working in pairs) produced better quality SRS documents with less ambiguity and consistency in obtained requirements as compared to the control group. The same holds for requirement completeness. The rationale behind the experimental group's enhancement in SRS document quality is comprehensible since having two pairs of eyes is more effective than one in identifying issues and conducting real-time quality assurance of the work at hand.

IV. THREATS TO VALIDITY

Although our experimental findings favored pair documentation, it is imperative to recognize the potential impediments that may affect the interpretation of these results. This section discusses four types of threats that may compromise the validity of our results.

A. INTERNAL VALIDITY

The factors that can affect external validity can be divergent personalities, creativity, competency, and level of intelligence in different students can affect the results. Similarly, interest in programming languages, previous knowledge,

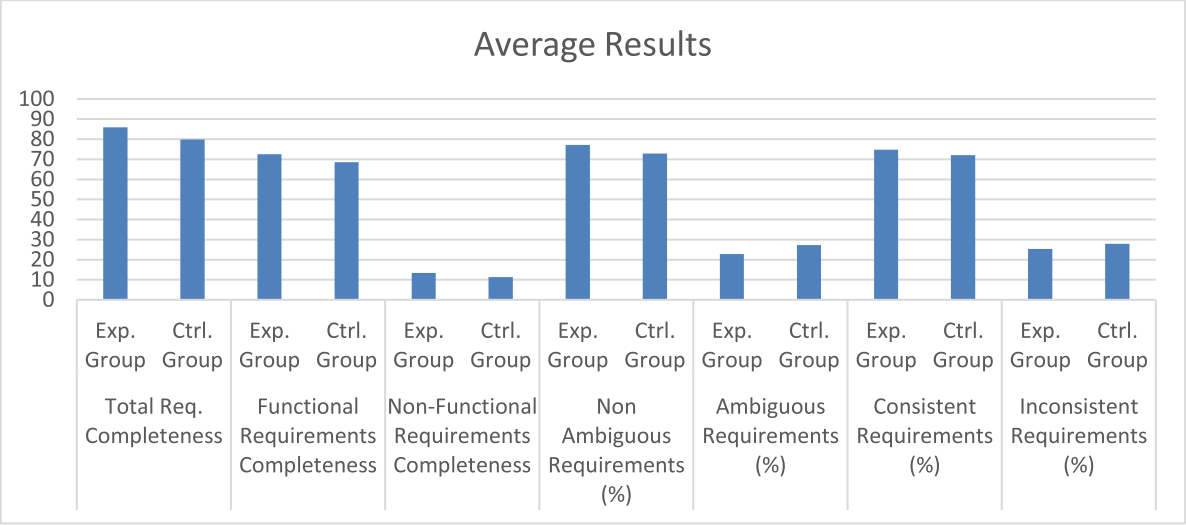

FIGURE 8. Comparison of average results from all the teams of both groups (Experimental and control groups).

and relevant experience can also impact the overall performance. To mitigate the above-mentioned threats, we formed teams by selecting the subjects from the same course/section and the same learning level (8th semester). We excluded the subjects with work experience or if they have done internships in industry.

B. EXTERNAL VALIDITY

The smaller number of teams can have an impact on external validity which is why we used a reasonable number of (i.e. 10) teams in each group to avoid this threat. Similarly, the use of different projects for two groups can influence the results. We avoided this threat by selecting the same project for both groups. Involving the students in a controlled experiment can be considered a threat to external validity.

C. CONSTRUCT VALIDITY

Construct Validity: We measured the completeness of the requirements, which is a subjective measure. To increase the objectivity of the evaluation, all the requirements' documents underwent two independent reviews; whenever they differed, the reviewers accomplished together a joint review of the documentation.

D. CONCLUSION VALIDITY

The assessment of results based on individual teams can cause a threat to conclusion validity. To mitigate this threat, we used the measure of central tendency i.e. mean/average so that the overall results of each group can be assessed and compared.

V. RESEARCH IMPLICATIONS

The outcomes of this research have different implications. The following sections discuss what these implications mean for three key stakeholders i.e. practitioners, educators, and researchers.

A. IMPLICATIONS FOR PRACTITIONERS

As the results go in favor of pair documentation, this implies that the concept of pair can be applied in the requirements engineering phases in software houses to produce better quality SRS documents, team cohesion, adaptability in changing requirements, knowledge sharing, and better collaboration.

B. IMPLICATIONS FOR EDUCATORS

Our results indicate that the teams using the pair documentation approach achieved better performance. This suggests that pair documentation could be effectively used as a pedagogical tool for improving learning outcomes, increased engagement, teamwork, better communication, and collaboration in requirements engineering courses in the software engineering stream during term projects and group-based assignments.

C. IMPLICATIONS FOR RESEARCHERS

This research is directed at the concept of pair documentation. The researchers can evaluate the concept with diverse experimental settings, different types of ambiguities, and other activities of requirements engineering. It would also be worthwhile to assess the concept of pairing on the other phases of the SDLC.

VI. CONCLUSION

This paper aims to study the significance of pair documentation, i.e., an unconventional requirements documentation approach which involves two people working on the same SRS simultaneously as compared to the traditional documentation approach. This is an empirical study conducted among two groups i.e. the control group and the experimental group which involves ten teams in each group. It has shown that the benefits of working in pairs apply not just to the other phases of the SDLC, i.e. design, programming, and testing but also to documentation. The results of the experiments reveal that

the teams using pair documentation, i.e., the experimental group, were more productive in terms of completeness of requirements i.e. they gathered more requirements in the given time. Moreover, the experimental group produced a better quality of requirements than the control group with less ambiguous and more consistent requirements. This research can be extended in several ways. Firstly, it can be replicated with different experimental settings e.g. different application domains, a greater number of subjects, a bigger project size, a higher experimental level, etc. This would also be worthwhile to explore the impact of pair documentation on other types of ambiguities as explored the lexical ambiguity e.g. semantic ambiguity, synthetic ambiguity, referential ambiguity, syntactic ambiguity, etc. Similarly, it would also be interesting to explore the impact of pair documentation on other activities of requirements engineering e.g. elicitation, analysis, validation, and documentation. To increase the reliability of the research, we intend to involve experienced, practicing requirements engineers and business analysts in the experiment.

ACKNOWLEDGMENT

Large language Models had been used to improve the quality of the writing. After preparing the first draft, the authors used LLMs to review the manuscript, enhance the literature and remove the typos. An early version of this manuscript had been shared, in 2023, and is available online [41].

REFERENCES

[1] X. Wang, L. Zhao, Y. Wang, and J. Sun, "The role of requirements engineering practices in agile development: An empirical study," in *Proc. 1st Asia–Pacific Requirements Eng. Symp.*, Auckland, New Zealand, Jan. 2014, pp. 195–209.

[2] R. Berntsson-Svensson and A. Aurum, "Successful software project and products: An empirical investigation," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng.*, New York, NY, USA, Sep. 2006, pp. 144–153.

[3] M. Rizwan Jameel Qureshi, S. Abo Alshamat, and F. Sabir, "Significance of the teamwork in agile software engineering," 2014, *arXiv:1408.6130*.

[4] E. Stephen and E. Mit, "Evaluation of software requirement specification based on IEEE 830 quality properties," *Int. J. Adv. Sci., Eng. Inf. Technol.*, vol. 10, no. 4, pp. 1396–1402, Aug. 2020.

[5] L. Jelai, E. Mit, and S. F. Samson Juan, "Knowledge representation framework for software requirement specification," *Int. J. Adv. Sci., Eng. Inf. Technol.*, vol. 10, no. 5, pp. 1846–1851, Oct. 2020.

[6] M. P. S. Bhatia, A. Kumar, R. Beniwal, and T. Malik, "Ontology driven software development for automatic detection and updation of software requirement specifications," *J. Discrete Math. Sci. Cryptogr.*, vol. 23, no. 1, pp. 197–208, Jan. 2020.

[7] C. Ribeiro and D. Berry, "The prevalence and severity of persistent ambiguity in software requirements specifications: Is a special effort needed to find them?" *Sci. Comput. Program.*, vol. 195, Sep. 2020, Art. no. 102472.

[8] Q. Zhi and S. Morisaki, "An evaluation of value-oriented review for software requirements specification," *Comput. Syst. Sci. Eng.*, vol. 37, no. 3, pp. 443–461, 2021.

[9] R. R. R. Merugu and S. R. Chinnam, "Automated cloud service based quality requirement classification for software requirement specification," *Evol. Intell.*, vol. 14, no. 2, pp. 389–394, Jun. 2021.

[10] N. Qamar, H. M. Kiran, F. Ahmad, S. Saeed, and B. Abid, "Software engineering fresh graduates career choices and software industry demands: An empirical analysis," *Int. J. Comput. Sci. Netw. Secur.*, vol. 20, no. 12, p. 72, Dec. 2020.

[11] M. Hasnain, I. Ghani, S. Ryul Jeong, M. Fermi Pasha, S. Usman, and A. Abbas, "Empirical analysis of software success rate forecasting during requirement engineering processes," *Comput., Mater. Continua*, vol. 74, no. 1, pp. 783–799, 2023.

[12] Chaos Rep. *Standish Group Int.* Accessed: Jan. 5, 2022. [Online]. Available: https://www.standishgroup.com/sample_research_files/CHAOSReport2015-Final.pdf

[13] N. J. Kipyegen and W. Korir, "Importance of software documentation," *Int. J. Comput. Sci. Issues*, vol. 10, no. 5, pp. 223–228, 2013.

[14] A. Brkić. *The Importance of Documentation in Software Development*. Accessed: May 25, 2022. [Online]. Available: https://serengetitech.com/business/the-importance-of-documentation-in-software-development/

[15] S. Ambler, *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*, 1st ed., Hoboken, NJ, USA: Wiley, 2002, ch. 6, sec. 2, pp. 64–68.

[16] M. R. R. Ramesh and C. S. Reddy, "Metrics for software requirements specification quality quantification," *Comput. Electr. Eng.*, vol. 96, Dec. 2021, Art. no. 107445.

[17] S. Jp, V. K. Menon, K. Soman, and A. K. R. Ojha, "A non-exclusive multi-class convolutional neural network for the classification of functional requirements in AUTOSAR software requirement specification text," *IEEE Access*, vol. 10, pp. 117707–117714, 2022.

[18] M. A. Jubair, S. A. Mostafa, A. Mustapha, M. A. Salamat, M. H. Hassan, M. A. Mohammed, and F. T. AL-Dhief, "A multi-agent K-means with case-based reasoning for an automated quality assessment of software requirement specification," *IET Commun.*, Dec. 2022.

[19] Z. Wang, J.-S. Pan, Q. Chen, and S. Yang, "BiLSTM-CRF-KG: A construction method of software requirements specification graph," *Appl. Sci.*, vol. 12, no. 12, p. 6016, Jun. 2022.

[20] R. Al-Msie'deen, A. H. Blasi, and M. A. Alsuwaiket, "Constructing a software requirements specification and design for electronic IT news magazine system," 2021, *arXiv:2111.01501*.

[21] M. Stella, F. Biscione, and M. Garzuoli, "Pair programming and other agile techniques: An overview and a hands-on experience," in *Proc. 4th Int. Conf. Softw. Eng. Defence Appl.*, Rome, Italy, Jan. 2016, pp. 87–101.

[22] T. Kude, S. Mithas, C. T. Schmidt, and A. Heinzl, "How pair programming influences team performance: The role of backup behavior, shared mental models, and task novelty," *Inf. Syst. Res.*, vol. 30, no. 4, pp. 1145–1163, Dec. 2019.

[23] J. Chong and R. Siino, "Interruptions on software teams: A comparison of paired and solo programmers," in *Proc. 20th anniversary Conf. Comput. Supported Cooperat. Work*, Banff, AB, Canada, Nov. 2006, pp. 29–38.

[24] P. Baheti, E. F. Gehringer, and D. Stotts, "Exploring the efficacy of distributed pair programming," in *Proc. Conf. Extreme Program. Agile Methods*, Copenhagen, Denmark. Cham, Switzerland: Springer, Jan. 2002, pp. 208–220.

[25] H. Hulkko and P. Abrahamsson, "A multiple case study on the impact of pair programming on product quality," in *Proc. 27th Int. Conf. Softw. Eng.*, St. Louis, MO, USA, 2005, pp. 495–504.

[26] L. Williams. *Pair Programming*. North Carolina State Univ. Accessed: Aug. 18, 2022. [Online]. Available: http://collaboration.csc.ncsu.edu/laurie/Papers/ESE%20WilliamsPairProgramming_V2.pdf

[27] R. A. Vera, M. M. Mata, J. D. Mendoza, and J. U. Pech, "Explorando la influencia de los roles de Belbin en la especificació n de requisitos de software," *Rev. Ibérica Sist. Tecnol. Inf.*, vol. 36, pp. 34–49, Mar. 2020.

[28] K. M. Lui and K. C. C. Chan, "Pair programming productivity: Novice–novice vs. Expert–expert," *Int. J. Hum.-Comput. Stud.*, vol. 64, no. 9, pp. 915–925, Sep. 2006.

[29] Ö. Demir and S. S. Seferoglu, "A comparison of solo and pair programming in terms of flow experience, coding quality, and coding achievement," *J. Educ. Comput. Res.*, vol. 58, no. 8, pp. 1448–1466, Jan. 2021.

[30] N. Qamar and A. A. Malik, "Evaluating the impact of pair testing on team productivity and test case quality—A controlled experiment," *Pak. J. Eng. Appl. Sci.*, vol. 25, pp. 80–88, Oct. 2019.

[31] G. Canfora, A. Cimitile, F. Garcia, M. Piattini, and C. A. Visaggio, "Performances of pair designing on software evolution: A controlled experiment," in *Proc. Conf. Softw. Maintenance Reengineering (CSMR)*, Bari, Italy, 2006, pp. 1–8.

[32] R. Swamidurai and U. Kannan, "Impact of pairing on various software development phases," in *Proc. ACM Southeast Regional Conf.*, New York, NY, USA, Mar. 2014, pp. 1–6.

[33] N. Qamar and A. A. Malik, "Birds of a feather gel together: Impact of team homogeneity on software quality and team productivity," *IEEE Access*, vol. 7, pp. 96827–96840, 2019.
[34] N. Qamar and A. A. Malik, "Determining the relative importance of personality traits in influencing software quality and team productivity," *Comput. Informat.*, vol. 39, no. 5, pp. 994–1021, 2020.
[35] N. Qamar and A. A. Malik, "A quantitative assessment of the impact of homogeneity in personality traits on software quality and team productivity," *IEEE Access*, vol. 10, pp. 122092–122111, 2022.
[36] A. K. Massey, R. L. Rutledge, A. I. Antón, and P. P. Swire, "Identifying and classifying ambiguity for regulatory requirements," in *Proc. IEEE 22nd Int. Requirements Eng. Conf. (RE)*, Karlskrona, Sweden, Aug. 2014, pp. 83–92.
[37] Q. Zhi, W. Pu, J. Ren, and Z. Zhou, "A defect detection method for the primary stage of software development," *Comput., Mater. Continua*, vol. 74, no. 3, pp. 5141–5155, 2023.
[38] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Automated handling of anaphoric ambiguity in requirements: A multi-solution study," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, Pittsburgh, PA, USA, May 2022, pp. 187–199.
[39] M. S. Husain, "Study of computational techniques to deal with ambiguity in SRS documents," in *Computational Intelligence in Software Modeling*, vol. 1, 1st ed., Berlin, Germany: De Gruyter, 2022, ch. 8, pp. 107–120.
[40] A. Nigam, N. Arya, B. Nigam, and D. Jain, "Tool for automatic discovery of ambiguity in requirements," *Int. J. Comput. Sci. Issues*, vol. 9, no. 5, p. 350, 2012.
[41] N. Qamar, N. Sabahat, A. Mashmool, and A. Mosavi, "Evaluating the impact of pair documentation on requirements quality and team productivity," 2023, *arXiv:2304.14255*.

NOSHEEN QAMAR received the Ph.D. degree in computer science from the National University of Computer and Emerging Sciences (FAST-NUCES), Lahore, Pakistan. She is currently an Assistant Professor with the Department of Software Engineering, University of Management and Technology, Lahore. Before joining the academia, in 2015, she gained seven years of industry experience as a Software Development Manager. Her research interests include software engineering, design patterns, software project management, software teams, empirical software engineering, and software requirements engineering.

NOSHEEN SABAHAT received the Ph.D. degree in computer software engineering from the National University of Sciences and Technology (NUST), Islamabad, Pakistan. She is currently an Assistant Professor with the Forman Christian College (A Chartered University), FCCU, Lahore, Pakistan. She received the Pakistan's Higher Education Commission (HEC) Indigenous Scholarship for M.S. leading to Ph.D. She has authored several research publications in internationally reputed journals and international conferences. Her research interests include software project management, usability evaluation, software sizing, requirements engineering, and empirical software engineering.

AMIR MOSAVI received the degree in engineering from Kingston University London and the Ph.D. degree in applied informatics. He has held a Postdoctoral Research Fellowship with the Norwegian University of Science and Technology, Bauhaus University Weimar, Germany, and Oxford Brookes University, U.K. He has also been a Visiting Researcher with the Queensland University of Technology, TU Dresden, and the Norwegian University of Life Sciences. He is currently a Data Scientist. Most recently, he served as a Guest Professor with the German Research Center for Artificial Intelligence. His work has been widely published in international peer-reviewed journals. Since 2019, he has been listed among the top 2% of scientists worldwide. His research interest includes applied machine learning. He has been recognized with numerous prestigious awards and fellowships, including the Green-Talent Award, the Alexander von Humboldt Award, the UNESCO Young Scientist Award, the Alain Bensoussan Fellowship, the GO STYRIA Award, the Dora Plus and Estophilus Fellowships (Estonia), the Future Talent TU Darmstadt Award, the World Academy of Sciences UNESCO Award, the Marie Curie Fellowship, the Endeavour-Australia Leadership Award, the Talented Young Scientist Award, and the European Research Consortium for Informatics and Mathematics Fellowship.